



纹理概述

- 纹理是一种普遍存在的视觉现象，目前对于纹理的精确定义还未形成统一认识，多根据应用需要做出不同定义.
- 两种较常采用的定义：
 - ◆ 定义1 按一定规则对元素或基元进行排列所形成的重复模式.
 - ◆ 定义2 如果图像函数的一组局部属性是恒定的，或者是缓变的，或者是近似周期性的，则图象中的对应区域具有恒定的纹理

纹理的特征与分类- 纹理是区域属性，并且与图像分辨率（或称尺度，resolution or scale）密切相关
重复性、规则性、周期性、方向性、随机性



纹理概述

纹理分析：研究纹理图像的特性

- ◆ 纹理分类：从给定的一组纹理集中识别给定的纹理区域，或作为重要特征帮助识别物体
- ◆ 纹理分割：把图片分成不同的部分，每部分内部具有相近的纹理。
- ◆ 从纹理恢复形状：由图像纹理恢复表面的方向和表面形状。

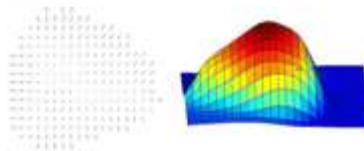
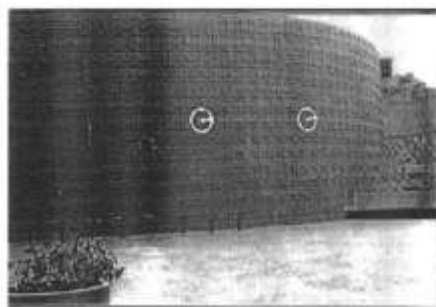
纹理合成：利用小样本构造大纹理

- ◆ 基于点合成：每次合成一个像素点。
- ◆ 基于块合成：每次合成一个像素块。



从纹理恢复形状

- 利用纹理随位置的变形估计形状



Pics from A. Loh: <http://www.csse.uwa.edu.au/~angie/phdpics1.html>



基于纹理分类



基于样图的纹理合成

1. 逐像素方式 (Efros & Leung)

(1) 在待填充像素位置(x,y)

- 取一定大小($S \times S$)的邻域B
- 在样图中选取N个同样大小($S \times S$)的图像块 $\{A_i\}$, $i=1, \dots, N$
- 在 $\{A_i\}$ 中选择寻找与B最相似图像块, 记为 A_0
- 将 A_0 中心的像素值填充到(x,y)

邻域大小

相似性计算

(2) 填充下一个位置, 直至所有区域填充完毕

18

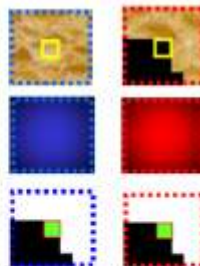


基于样图的纹理合成

1. 逐像素方式

– 相似性计算, 选择最匹配的样图

- 根据离中心的距离高斯加权
- 仅使用有纹理的区域

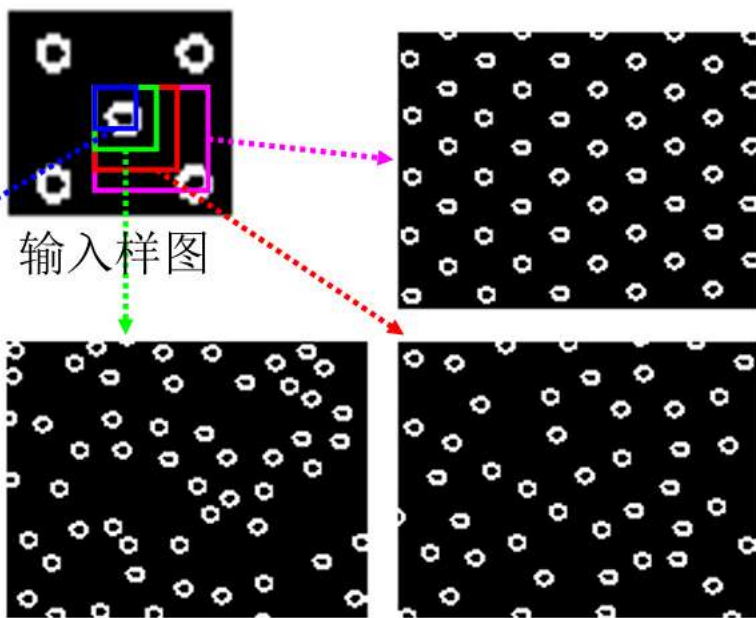


最小平方距离 $SSD(I_1, I_2) = \sum_{x,y} (I_1(x,y) - I_2(x,y))^2$

2 基于样图的纹理合成

1. 逐像素方式

— 邻域大小的影响

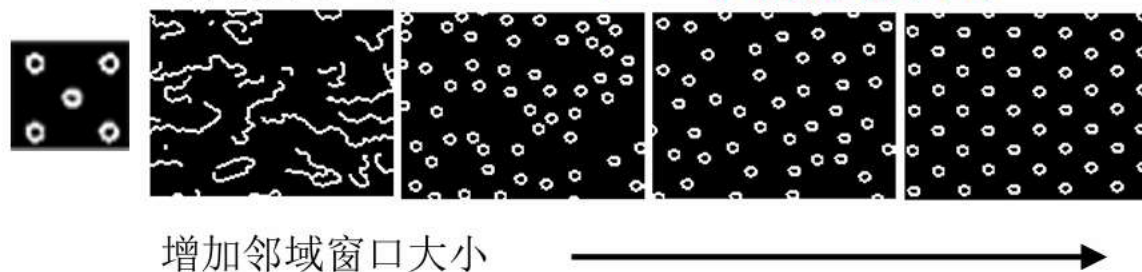


22

2 基于样图的纹理合成

1. 逐像素方式

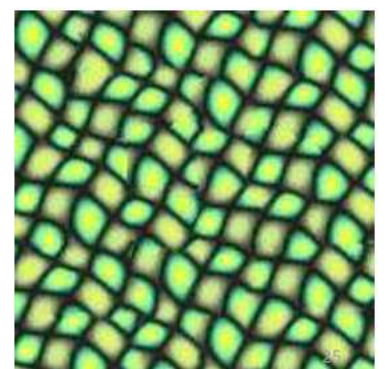
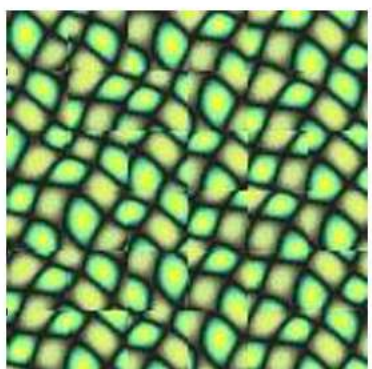
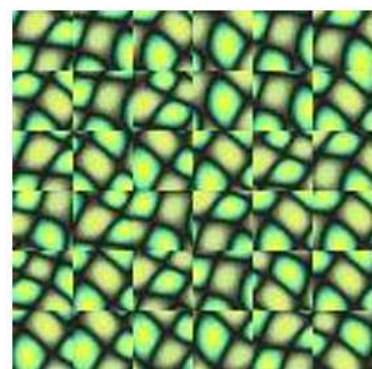
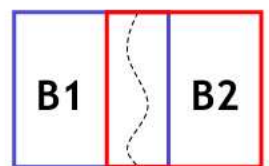
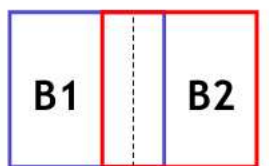
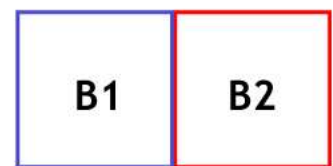
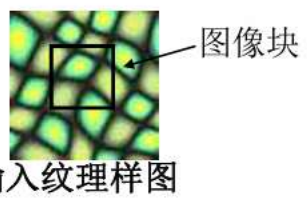
图像块方式



窗口大小选择

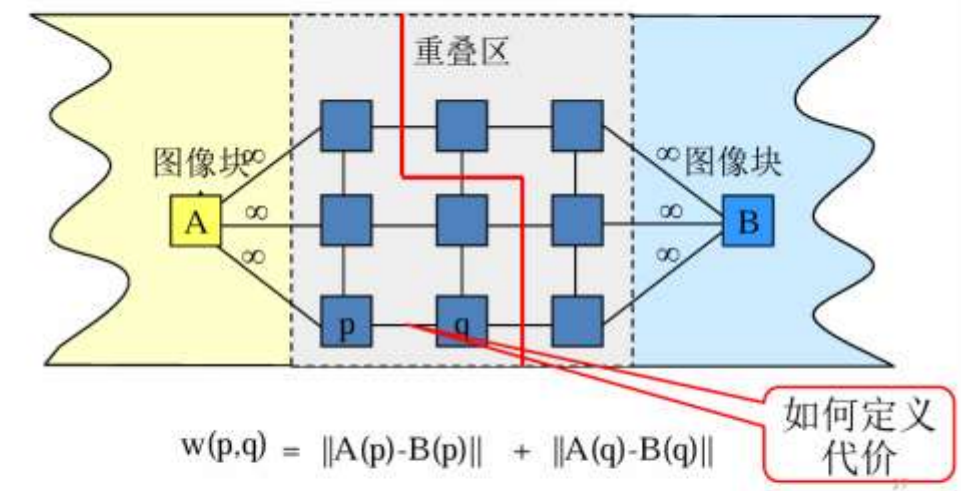
- (1) 足够大，包含尽量多的纹理，（对于规则纹理）至少包括一个纹理重复单元
- (2) 不能太大，使得样图上采样数目比较少

24



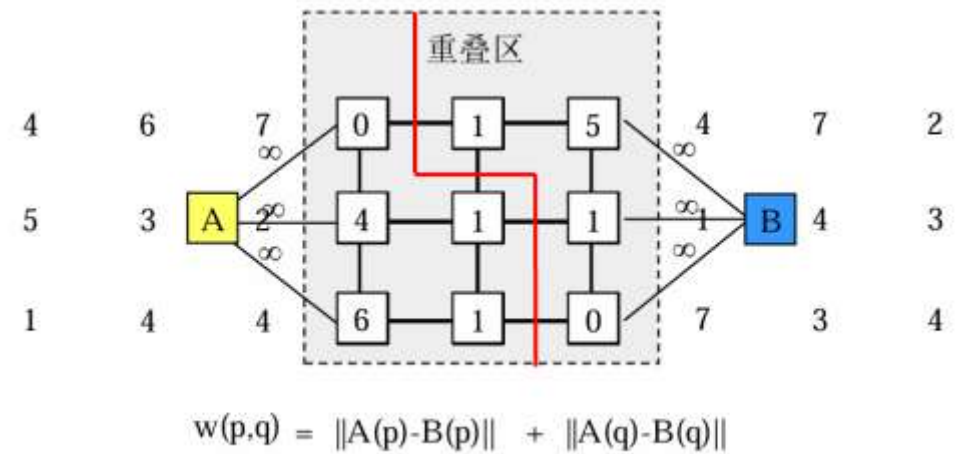
基于样图的纹理合成

2. 图像块方式



基于样图的纹理合成

2. 图像块方式



一、基于点的纹理合成 (Pixel-based / Point-based Synthesis)

核心思想：把纹理生成看作是一个马尔可夫随机场（MRF）过程。每次只生成一个像素，通过对比该像素周围的邻域窗口与原图的相似度，来决定当前像素填什么颜色。

✔ 优点：

- 灵活性极高：**因为是逐像素生成的，它可以非常灵活地适应任何形状的边界或空洞。无论需要修复的区域多么崎岖、狭窄，算法都能顺着边缘一点点“生长”进去。
- 对随机/微观纹理效果极佳：**对于那些没有固定宏观结构、纯粹由微小颗粒组成的纹理（如沙地、粗糙的水泥墙、草坪），逐像素法能够完美还原那种微观的随机性和颗粒感，几乎看不出任何拼接痕迹。

✘ 缺点：

- 计算效率极其低下（致命伤）：**生成一张新图需要经历千万次搜索。假设你要生成一个 1000×1000 的图像，算法必须在原图里进行 100 万次完整的滑动窗口匹配，计算耗时往往让人难以忍受。
- 容易发生“垃圾衍生” (Garbage Growing)：**算法存在严重的误差累积问题。如果某一个像素生成错了（比如把边缘的杂色当成了纹理），下一个像素会把这个错误像素作为参考，导致错误被不断放大，最终长出一大片毫无意义的模糊色块。
- 容易丢失大尺度的语义结构：**正如之前提到的，如果邻域窗口设置得不够大，算法就如同“盲

- 容易产生重复感和“打补丁”感：**如果输入的样图非常小，算法只能反复使用那几个有限的图像块。当合成大面积纹理时，人眼很容易察觉到某种图案在周期性地重复出现，看起来像劣质壁纸。
- 对边界的处理要求极高：**虽然有最小代价路径算法来隐藏缝隙，但在某些高度结构化、线条分明的纹理（比如笔直的铁丝网、规则的棋盘格）中，如果两个块没能完美对齐，拼接处的断裂感会非常刺眼。
- 缺乏形状适应性：**当遇到边缘极其狭小复杂的待修复区域时，塞进去一个巨大的方形图像块往往显得非常笨拙。

- 容易丢失大尺度的语义结构：**正如之前提到的，如果邻域窗口设置得不够大，算法就如同“盲人摸象”，只能画出砖块的局部边缘，却无法画出一整面整齐排列的砖墙。

二、基于图像块的纹理合成 (Patch-based Synthesis)

核心思想：放弃逐个像素的挣扎，直接从原图里切下一整块“图像块”，像贴瓷砖一样将其复制到目标区域，然后利用“最小代价边界”算法处理重叠部分的缝隙（如 Image Quilting）。

✔ 优点：

- 速度呈指数级提升：**一次直接搞定几十甚至上百个像素，全局搜索和匹配的次数大幅减少，运算效率远超逐像素法。
- 完美保留宏观语义和结构（最大优势）：**因为复制的是原图中真实存在的图像块，所以块内部的纹理结构（如一整朵花、一个完整的圆环、一根完整的树枝）绝对原汁原味，不会发生任何扭曲或破碎。
- 彻底消灭“垃圾衍生”：**误差不会在像素之间无限累积，最多只会在两个图像块的拼接边界处产生极小范围的伪影。

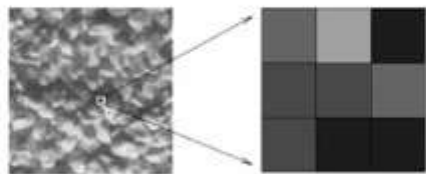
✘ 缺点：

- 容易产生重复感和“打补丁”感：**如果输入的样图非常小，算法只能反复使用那几个有限的图像块。当合成大面积纹理时，人眼很容易察觉到某种图案在周期性地重复出现，看起来像劣

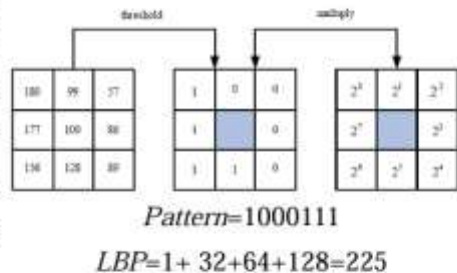


局部二值模式 (LBP)

■ 基本的LBP算子: 3×3 的矩形块, 有1个中心像素和8个邻域像素对应于9个灰度值。



■ 特征值: 中心像素的灰度值为阈值, 将其邻域的8个灰度值与阈值相比较, 大于中心灰度值的像素由1表示, 反之由0表示。然后根据顺时针方向读出8个二进制值。



43



局部二值模式 (LBP)

■ 在不丢失信息的前提下, 将邻域像素的灰度值分别减去邻域中心的灰度值, 局部纹理特征则可表示为

$$T = t(g_c, g_0 - g_c, \dots, g_{p-1} - g_c)$$

■ 假设各个差值与 g_c 相互独立, 则上式可分解为:

$$T \approx t(g_c) t(g_0 - g_c, \dots, g_{p-1} - g_c)$$

■ $t(g_c)$ 代表图像的亮度值, 与图像局部纹理特征无关, 所以可将纹理特征表示为差值的函数:

$$T \approx t(g_0 - g_c, \dots, g_{p-1} - g_c)$$

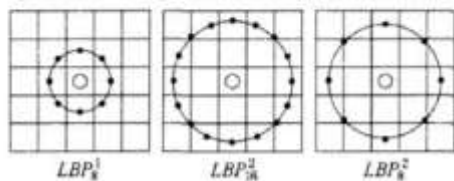
■ 这一P维差值函数记录了邻域中每个像素的纹理模式

44



局部二值模式 (LBP)

在某一灰度图像中, 定义一个半径为 R ($R>0$) 的圆环形邻域, p ($p>0$) 个邻域像素均匀分布在圆周上。设该邻域中心像元的纹理为 T , 则 T 可以用该邻域中 $P+1$ 个像素的函数来定义, 即

$$T = t(g_c, g_0, \dots, g_{p-1})$$


$$x_p = x_c + R \cos(2\pi p/P)$$

$$y_p = y_c - R \sin(2\pi p/P)$$



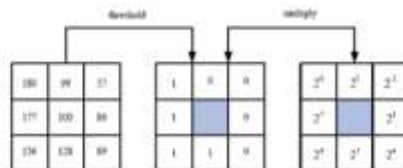
局部二值模式 (LBP)

■ 为了使定义的纹理不受灰度值单调变换的影响, 只考虑差值的符号:

$$T \approx t(s(g_0 - g_c), \dots, s(g_{p-1} - g_c)), \text{ 其中 } s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

■ 表征局部纹理特征的LBP值可以表示为:

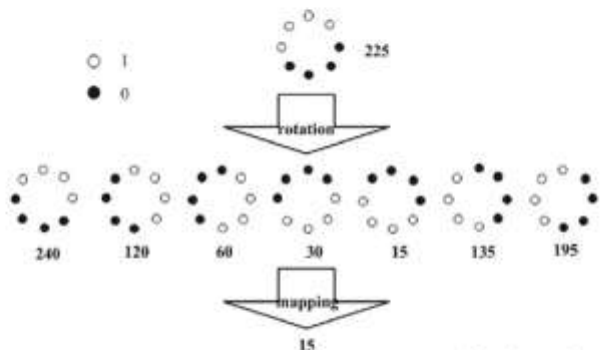
$$LBP_{p,R} = \sum_{i=0}^{p-1} s(g_i - g_c) 2^i$$





局部二值模式 (LBP)

- LBP具有平移不变性
- LBP具有旋转不变性?



- 等价模式 (Uniform pattern, 1-0、0-1最多2次)
混合模式

46



局部二值模式 (LBP)

- 特点: 经阈值化后的二值矩阵可看成一个**二值纹理模式**, 用来刻画邻域内像素点的灰度相对中心点的变化情况。因为人类视觉系统对纹理的感知与平均灰度(亮度)无关, 而局部二值模式方法注重像素灰度的变化, 所以它符合人类视觉对图像纹理的感知特点。

- 用途: 工业视觉检测、图像检索、人脸识别、表情识别、运动目标实时跟踪等

2. 如何实现“旋转不变性” (Rotation Invariant LBP)?

看图片里那个向下指着 15 的 mapping (映射) 箭头。

解决思路非常巧妙且简单粗暴: 既然同一个特征旋转后会产生 8 个不同的数字, 那我们就把这 8 个数字全部归为一类, 并统一用它们之中最小的那个数字来代表它们。

在这 8 个数字 (225, 240, 120, 60, 30, 15, 135, 195) 里, 最小的是 15 (二进制 00001111)。

所以, 以后算法无论读到这 8 个数字里的哪一个, 都会强制把它映射 (记录) 为 15。这样, 即使图像发生了旋转, 提取出的 LBP 特征值也永远保持一致了。

3. 什么是“等价模式” (Uniform Pattern)?

这是图片最下方那句话的意思。它是为了解决 LBP 特征维度过高的问题 (也就是所谓的降维)。

- 什么是跳变? 顺着圆周把这 8 个二进制位读一圈 (最后一位要和第一位连起来看), 从 0 变成 1, 或者从 1 变成 0, 就叫一次跳变。
- 什么是等价模式? 规定跳变次数 ≤ 2 次的二进制序列, 就是等价模式。
 - 比如 00001111: 从 0 到 1 跳了 1 次, 末尾的 1 回到开头的 0 又跳了 1 次, 总共跳 2 次。它是等价模式。
 - 比如 01010101: 0 和 1 交替, 总共跳了 8 次。它不是等价模式 (被称为混合模式)。

为什么要这么做?

经过大量统计发现, 自然界的图像中, 绝大多数 (通常占 90% 以上) 的局部纹理都是平缓的区域、简单的边缘或角点, 它们对应的 LBP 值跳变次数都很少。那些跳变次数 ≥ 3 的模式, 往往是高频的噪点。

对于 8 个采样点来说, 原本有 $2^8 = 256$ 种可能的值。经过等价模式过滤后, 符合跳变 ≤ 2 的模式只有 58 种。我们把所有大于 2 次跳变的混合模式全部扔进一个“杂项”垃圾桶 (算作第 59 种)。这样就把特征向量的维度从 256 维大幅降到了 59 维, 不仅计算更快, 还能起到抗噪的作用。



■ 基于LBP的人脸识别

- ◆ 计算人脸图像LBP响应
- ◆ 人脸图片分为若干个相等的矩形区域



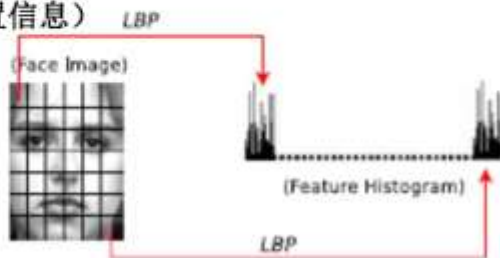
(a)

50



■ 基于LBP的人脸识别

- ◆ 计算人脸图像LBP响应
- ◆ 人脸图片分为若干个相等的矩形区域
- ◆ 在每个区域计算LBP直方图，再将它们串联起来
(不仅保留原图像的统计信息，同时从直方图上也可以看出位置信息)
- ◆ 计算相似性



■ 基于LBP的人脸识别

- ◆ 由于图片上各个区域包含的有效信息不一样，所以可以对各个区域赋予不同的权重，其中一种方法是：将图像分为若干个区域后，依次取用每个区域对图像进行识别，识别率在0.2以下的权重为0，识别率在0.8以上的权重为2，识别率在0.9以上的权重为4，其余的权重设为1。



(a)



(b)

52

- 设下图所示的为一个像素的邻域，先要借助（8，2）的圆邻域计算LBP，写出所得到的二进制标号和十进制标号。如果借助（16，2）的圆邻域呢？

0	0	0	1	2
1	1	0	1	1
2	2	1	0	0
1	1	0	2	0
0	0	1	0	1